МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»
Кафедра автоматизованих систем управління

ЗВІТ

про виконання лабораторної роботи №8

«Форми введення даних та навігація між екранами»

з дисципліни «Програмування мобільних додатків»

Виконав:
студент групи КН-2327Б
Багрій-Белз Андрій

Прийняв:
викладач кафедри АСУ

Львів — 2026

Лабораторна робота №8

Тема: Форми введення даних та навігація між екранами

Мета роботи:

Сформувати практичні навички створення форм введення даних у Flutter, навчитися використовувати віджети TextField, TextFormField, ElevatedButton, Form, TextEditingController, реалізовувати комплексну перевірку правильності введених даних (валідацію), а також організовувати міжвіконну навігацію з передачею типізованих параметрів між екранами за допомогою Navigator.

Завдання на лабораторну роботу:

1. Створити екран реєстрації RegisterScreen.

2. Реалізувати форму введення даних за допомогою віджета Form.

3. Додати щонайменше такі обов'язкові поля введення: ім’я користувача, дата народження, email, пароль, підтвердження пароля.

4. Для полів введення використати віджети TextFormField.

5. Реалізувати перевірку правильності введених даних: усі поля не повинні бути порожніми; ім’я користувача повинно містити не менше 2–3 символів; дата народження має мати перевірку на коректний формат дати; email повинен містити символ @; пароль повинен містити не менше 6 символів; поле підтвердження пароля повинно збігатися з паролем.

6. Використати екземпляри TextEditingController для зчитування введених значень та своєчасного їх звільнення у dispose().

7. Додати стилізовану кнопку «Зареєструватися» (ElevatedButton).

8. Реалізувати навігацію з RegisterScreen на HomeScreen.

9. Передати на екран HomeScreen ім’я користувача та додаткові дані реєстрації.

10. Відобразити на екрані HomeScreen текст привітання, наприклад: «Вітаємо, Ім’я!».

11. Використати Navigator.push() або Navigator.pushReplacement() для переходу між екранами.

12. Оформити інтерфейс екранів із використанням Scaffold, AppBar, Padding, Column.

13. Перевірити, що перехід на HomeScreen виконується лише після успішної валідації форми (_formKey.currentState!.validate()).

1 Написання коду-розв’язку завдання

У процесі виконання лабораторної роботи було створено мобільний застосунок реєстрації та авторизації registration_app. Архітектура проєкту розділена на модулі: lib/main.dart (точка входу та конфігурація Material 3 теми), lib/screens/register_screen.dart (екран форми реєстрації RegisterScreen із валідацією, контролерами та вибором дати через DatePicker), lib/screens/home_screen.dart (головний екран HomeScreen із відображенням персонального привітання та карток зареєстрованого профілю), а також test/widget_test.dart (автоматизовані віджет-тести для перевірки всіх правил валідації та міжвіконної навігації).

Лістинг файлу lib/main.dart (Конфігурація теми та точка входу):

```
import 'package:flutter/material.dart';
import 'screens/register_screen.dart';

void main() {
  runApp(const RegistrationApp());
}

class RegistrationApp extends StatelessWidget {
  const RegistrationApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Форми та навігація — Лабораторна №8',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        brightness: Brightness.dark,
        scaffoldBackgroundColor: const Color(0xFF0D0F13),
        colorScheme: const ColorScheme.dark(
          primary: Color(0xFFD4FF00),
          surface: Color(0xFF15181F),
          onSurface: Colors.white,
        ),
      ),
      home: const RegisterScreen(),
    );
  }
}
```

Лістинг файлу lib/screens/home_screen.dart (Головний екран HomeScreen):

```
import 'package:flutter/material.dart';

/// Головний екран HomeScreen (п. 8 ТЗ)
/// Отримує ім'я користувача (п. 9 ТЗ) та інші дані реєстрації
class HomeScreen extends StatelessWidget {
  final String username;
  final String? email;
  final String? birthDate;

  const HomeScreen({
    super.key,
    required this.username,
    this.email,
    this.birthDate,
  });

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: const Color(0xFF0D0F13),
      appBar: AppBar(
        title: const Text(
          'Головний екран',
          style: TextStyle(fontWeight: FontWeight.bold),
        ),
        centerTitle: true,
        backgroundColor: const Color(0xFF15181F),
        elevation: 0,
        leading: IconButton(
          icon: const Icon(Icons.arrow_back_ios_new, color: Color(0xFF8B949E)),
          tooltip: 'Назад',
          onPressed: () => Navigator.pop(context),
        ),
```

```
        actions: [
          IconButton(
            icon: const Icon(Icons.logout, color: Color(0xFFFF5252)),
            tooltip: 'Вийти з акаунта',
            onPressed: () {
              Navigator.pop(context);
            },
          ),
        ],
      ),
      body: SafeArea(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(20.0),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              const SizedBox(height: 10),

              // 1. Картка привітання (п. 10 ТЗ: «Вітаємо, Ім’я!»)
              Container(
                padding: const EdgeInsets.all(24.0),
                decoration: BoxDecoration(
                  gradient: const LinearGradient(
                    colors: [Color(0xFF1F2430), Color(0xFF15181F)],
                    begin: Alignment.topLeft,
                    end: Alignment.bottomRight,
                  ),
                  borderRadius: BorderRadius.circular(20),
                  border: Border.all(color: const Color(0xFFD4FF00), width: 1.5),
                  boxShadow: [
                    BoxStyle(
                      color: const Color(0xFFD4FF00).withValues(alpha: 0.15),
                      blurRadius: 20,
                      spreadRadius: 2,
                    ),
                  ],
                ),
                child: Column(
                  children: [
                    Container(
                      padding: const EdgeInsets.all(16),
                      decoration: BoxDecoration(
                        color: const Color(0xFFD4FF00).withValues(alpha: 0.15),
                        shape: BoxShape.circle,
                      ),
                      child: const Icon(
                        Icons.check_circle_rounded,
                        color: Color(0xFFD4FF00),
                        size: 54,
                      ),
                    ),
                    const SizedBox(height: 16),
                    Text(
                      'Вітаємо, $username! 🎉',
                      textAlign: TextAlign.center,
                      style: const TextStyle(
                        fontSize: 24,
                        fontWeight: FontWeight.bold,
                        color: Colors.white,
                        letterSpacing: 0.3,
                      ),
                    ),
                    const SizedBox(height: 8),
                    const Text(
                      'Реєстрація пройшла успішно! Валідацію всіх полів форми пройдено без
помилок.',
                      textAlign: TextAlign.center,
                      style: TextStyle(
```

```
                fontSize: 14,
                color: Color(0xFF8B949E),
                height: 1.4,
              ),
            ),
          ],
        ),
      ),

      const SizedBox(height: 24),

      // 2. Блок відображення переданих даних
      Container(
        padding: const EdgeInsets.all(20),
        decoration: BoxDecoration(
          color: const Color(0xFF15181F),
          borderRadius: BorderRadius.circular(16),
          border: Border.all(color: const Color(0xFF2A3142)),
        ),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Row(
              children: [
                Icon(Icons.badge_outlined, color: Color(0xFF4FC3F7), size: 22),
                SizedBox(width: 8),
                Expanded(
                  child: Text(
                    'Дані зареєстрованого профілю',
                    style: TextStyle(
                      fontSize: 16,
                      fontWeight: FontWeight.bold,
                      color: Colors.white,
                    ),
                  ),
                ),
              ],
            ),
            const Divider(color: Color(0xFF2A3142), height: 24),
            _buildDataRow(
              icon: Icons.person_outline,
              label: "Ім'я користувача:",
              value: username,
              highlight: true,
            ),
            if (email != null && email!.isNotEmpty) ...[
              const SizedBox(height: 12),
              _buildDataRow(
                icon: Icons.email_outlined,
                label: 'Електронна пошта:',
                value: email!,
              ),
            ],
            if (birthDate != null && birthDate!.isNotEmpty) ...[
              const SizedBox(height: 12),
              _buildDataRow(
                icon: Icons.cake_outlined,
                label: 'Дата народження:',
                value: birthDate!,
              ),
            ],
            const SizedBox(height: 12),
            _buildDataRow(
              icon: Icons.verified_user_outlined,
              label: 'Статус акаунта:',
              value: 'Активний / Верифікований',
            ),
          ],
```

```
              ),
            ),

            const SizedBox(height: 32),

            // 3. Кнопка виходу / повернення на екран реєстрації
            OutlinedButton.icon(
              style: OutlinedButton.styleFrom(
                foregroundColor: const Color(0xFFD4FF00),
                side: const BorderSide(color: Color(0xFFD4FF00), width: 1.5),
                padding: const EdgeInsets.symmetric(vertical: 16),
                shape: RoundedRectangleBorder(
                  borderRadius: BorderRadius.circular(12),
                ),
              ),
              icon: const Icon(Icons.arrow_back),
              label: const Text(
                'Повернутися до реєстрації',
                style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
              ),
              onPressed: () {
                Navigator.pop(context);
              },
            ),
          ],
        ),
      ),
    ),
  );
  }

  Widget _buildDataRow({
    required IconData icon,
    required String label,
    required String value,
    bool highlight = false,
  }) {
    return Row(
      children: [
        Icon(icon, size: 18, color: const Color(0xFF8B949E)),
        const SizedBox(width: 8),
        Text(
          label,
          style: const TextStyle(
            fontSize: 13,
            color: Color(0xFF8B949E),
          ),
        ),
        const SizedBox(width: 8),
        Expanded(
          child: Text(
            value,
            textAlign: TextAlign.end,
            overflow: TextOverflow.ellipsis,
            style: TextStyle(
              fontSize: 14,
              fontWeight: highlight ? FontWeight.bold : FontWeight.w600,
              color: highlight ? const Color(0xFFD4FF00) : Colors.white,
            ),
          ),
        ),
      ],
    );
  }
}

class BoxStyle extends BoxShadow {
  const BoxStyle({
```

```
    required super.color,
    required super.blurRadius,
    required super.spreadRadius,
  });
}
```

Лістинг файлу lib/screens/register_screen.dart (Екран реєстрації RegisterScreen):

```
import 'package:flutter/material.dart';
import 'home_screen.dart';

/// Екран реєстрації користувача RegisterScreen (п. 1 ТЗ)
class RegisterScreen extends StatefulWidget {
  const RegisterScreen({super.key});

  @override
  State<RegisterScreen> createState() => _RegisterScreenState();
}

class _RegisterScreenState extends State<RegisterScreen> {
  // Ключ форми для централізованої валідації (п. 2 ТЗ)
  final GlobalKey<FormState> _formKey = GlobalKey<FormState>();

  // TextEditingController для кожного поля форми (п. 6 ТЗ)
  final TextEditingController _usernameController = TextEditingController();
  final TextEditingController _birthDateController = TextEditingController();
  final TextEditingController _emailController = TextEditingController();
  final TextEditingController _passwordController = TextEditingController();
  final TextEditingController _confirmPasswordController =
      TextEditingController();

  // Стан видимості паролів
  bool _obscurePassword = true;
  bool _obscureConfirmPassword = true;

  @override
  void dispose() {
    // Обов'язкове звільнення ресурсів контролерів для запобігання витокам пам'яті
    _usernameController.dispose();
    _birthDateController.dispose();
    _emailController.dispose();
    _passwordController.dispose();
    _confirmPasswordController.dispose();
    super.dispose();
  }

  // Вибір дати народження за допомогою DatePicker
  Future<void> _selectBirthDate(BuildContext context) async {
    final DateTime now = DateTime.now();
    final DateTime initialDate = DateTime(2004, 1, 1);
    final DateTime firstDate = DateTime(1900);
    final DateTime lastDate = now;

    final DateTime? picked = await showDatePicker(
      context: context,
      initialDate: initialDate,
      firstDate: firstDate,
      lastDate: lastDate,
      helpText: 'Оберіть дату народження',
      cancelText: 'Скасувати',
      confirmText: 'Обрати',
      builder: (context, child) {
        return Theme(
          data: Theme.of(context).copyWith(
            colorScheme: const ColorScheme.dark(
```

```
              primary: Color(0xFFD4FF00),
              onPrimary: Color(0xFF0D0F13),
              surface: Color(0xFF15181F),
              onSurface: Colors.white,
            ),
            dialogTheme: const DialogThemeData(backgroundColor: Color(0xFF15181F)),
          ),
          child: child!,
        );
      },
    );

    if (picked != null) {
      final String formatted =
          '${picked.year}-${picked.month.toString().padLeft(2, '0')}-$
{picked.day.toString().padLeft(2, '0')}';
      setState(() {
        _birthDateController.text = formatted;
      });
    }
  }

  // Обробник натискання кнопки «Зареєструватися» (п. 7, 8, 13 ТЗ)
  void _submitForm() {
    // Перевірка валідації форми перед переходом (п. 13 ТЗ)
    if (_formKey.currentState!.validate()) {
      final String username = _usernameController.text.trim();
      final String email = _emailController.text.trim();
      final String birthDate = _birthDateController.text.trim();

      // Навігація на HomeScreen із передачею імені користувача (п. 8, 9, 11 ТЗ)
      Navigator.push(
        context,
        MaterialPageRoute(
          builder: (context) => HomeScreen(
            username: username,
            email: email,
            birthDate: birthDate,
          ),
        ),
      );
    } else {
      // Якщо є помилки, інформуємо користувача через SnackBar
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: const Text(
            'Будь ласка, виправте виділені помилки у формі!',
            style: TextStyle(
              color: Color(0xFF0D0F13),
              fontWeight: FontWeight.bold,
            ),
          ),
          backgroundColor: const Color(0xFFFF5252),
          behavior: SnackBarBehavior.floating,
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(10),
          ),
          duration: const Duration(seconds: 2),
        ),
      );
    }
  }

  // Допоміжна функція швидкого заповнення демо-даними
  void _fillDemoData() {
    setState(() {
      _usernameController.text = 'Андрій Багрій-Белз';
      _birthDateController.text = '2004-05-15';
```

```
    _emailController.text = 'andrii.bagriy@nulp.ua';
    _passwordController.text = 'flutter2026';
    _confirmPasswordController.text = 'flutter2026';
  });
}

@override
Widget build(BuildContext context) {
  // Оформлення з використанням Scaffold, AppBar, Padding, Column (п. 12 ТЗ)
  return Scaffold(
    backgroundColor: const Color(0xFF0D0F13),
    appBar: AppBar(
      title: const Text(
        'Реєстрація',
        style: TextStyle(fontWeight: FontWeight.bold),
      ),
      centerTitle: true,
      backgroundColor: const Color(0xFF15181F),
      elevation: 0,
      actions: [
        IconButton(
          tooltip: 'Заповнити тестовими даними',
          icon: const Icon(Icons.auto_fix_high, color: Color(0xFFD4FF00)),
          onPressed: _fillDemoData,
        ),
      ],
    ),
    body: SafeArea(
      child: SingleChildScrollView(
        padding: const EdgeInsets.symmetric(horizontal: 20.0, vertical: 16.0),
        child: Form(
          key: _formKey, // Прив'язка Form до _formKey (п. 2 ТЗ)
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              // Заголовок форми
              Center(
                child: Container(
                  padding: const EdgeInsets.all(14),
                  decoration: BoxDecoration(
                    color: const Color(0xFFD4FF00).withValues(alpha: 0.1),
                    shape: BoxShape.circle,
                    border: Border.all(
                      color: const Color(0xFFD4FF00).withValues(alpha: 0.4),
                    ),
                  ),
                  child: const Icon(
                    Icons.person_add_alt_1_rounded,
                    color: Color(0xFFD4FF00),
                    size: 38,
                  ),
                ),
              ),
              const SizedBox(height: 12),
              const Text(
                'Створіть свій обліковий запис',
                textAlign: TextAlign.center,
                style: TextStyle(
                  fontSize: 20,
                  fontWeight: FontWeight.bold,
                  color: Colors.white,
                ),
              ),
              const SizedBox(height: 4),
              const Text(
                'Заповніть усі обов’язкові поля для входу в систему',
                textAlign: TextAlign.center,
                style: TextStyle(fontSize: 13, color: Color(0xFF8B949E)),
```

```
                ),
                const SizedBox(height: 24),

                // 1. Поле: ім’я користувача (п. 3, 4, 5 ТЗ)
                TextFormField(
                  controller: _usernameController,
                  keyboardType: TextInputType.name,
                  style: const TextStyle(color: Colors.white),
                  decoration: _inputDecoration(
                    labelText: "Ім'я користувача",
                    hintText: 'Введіть ваше ім’я або нікнейм',
                    prefixIcon: Icons.person_outline,
                  ),
                  validator: (value) {
                    if (value == null || value.trim().isEmpty) {
                      return "Поле імені користувача не може бути порожнім";
                    }
                    if (value.trim().length < 3) {
                      return "Ім'я користувача повинно містити щонайменше 3 символи";
                    }
                    return null;
                  },
                ),
                const SizedBox(height: 16),

                // 2. Поле: дата народження (п. 3, 4, 5 ТЗ)
                TextFormField(
                  controller: _birthDateController,
                  readOnly: false,
                  keyboardType: TextInputType.datetime,
                  style: const TextStyle(color: Colors.white),
                  decoration: _inputDecoration(
                    labelText: 'Дата народження',
                    hintText: 'РРРР-ММ-ДД або ДД.ММ.РРРР',
                    prefixIcon: Icons.cake_outlined,
                    suffixIcon: IconButton(
                      icon: const Icon(
                        Icons.calendar_month_outlined,
                        color: Color(0xFFD4FF00),
                      ),
                      tooltip: 'Обрати дату з календаря',
                      onPressed: () => _selectBirthDate(context),
                    ),
                  ),
                  validator: (value) {
                    if (value == null || value.trim().isEmpty) {
                      return 'Введіть дату народження';
                    }
                    final trimmed = value.trim();
                    // Перевірка формату дати: РРРР-ММ-ДД або ДД.ММ.РРРР
                    final dateRegex = RegExp(
                      r'^((\d{4}[-/.]\d{2}[-/.]\d{2})|(\d{2}[-/.]\d{2}[-/.]\d{4}))$',
                    );
                    if (!dateRegex.hasMatch(trimmed)) {
                      return 'Некоректний формат дати (приклад: 2004-05-15 або 15.05.2004)';
                    }
                    return null;
                  },
                ),
                const SizedBox(height: 16),

                // 3. Поле: email (п. 3, 4, 5 ТЗ)
                TextFormField(
                  controller: _emailController,
                  keyboardType: TextInputType.emailAddress,
                  style: const TextStyle(color: Colors.white),
                  decoration: _inputDecoration(
                    labelText: 'Електронна пошта (Email)',
```

```
                    hintText: 'user@example.com',
                    prefixIcon: Icons.email_outlined,
                  ),
                  validator: (value) {
                    if (value == null || value.trim().isEmpty) {
                      return 'Введіть адресу електронної пошти';
                    }
                    final trimmed = value.trim();
                    if (!trimmed.contains('@')) {
                      return 'Email повинен містити символ @';
                    }
                    if (!trimmed.contains('.') || trimmed.indexOf('@') > trimmed.lastIndexOf('.'))
{
                      return 'Email повинен містити коректне доменне ім’я';
                    }
                    return null;
                  },
                ),
                const SizedBox(height: 16),

                // 4. Поле: пароль (п. 3, 4, 5 ТЗ)
                TextFormField(
                  controller: _passwordController,
                  obscureText: _obscurePassword,
                  style: const TextStyle(color: Colors.white),
                  decoration: _inputDecoration(
                    labelText: 'Пароль',
                    hintText: 'Не менше 6 символів',
                    prefixIcon: Icons.lock_outline,
                    suffixIcon: IconButton(
                      icon: Icon(
                        _obscurePassword
                            ? Icons.visibility_outlined
                            : Icons.visibility_off_outlined,
                        color: const Color(0xFF8B949E),
                      ),
                      onPressed: () {
                        setState(() {
                          _obscurePassword = !_obscurePassword;
                        });
                      },
                    ),
                  ),
                  validator: (value) {
                    if (value == null || value.isEmpty) {
                      return 'Введіть пароль';
                    }
                    if (value.length < 6) {
                      return 'Пароль повинен містити не менше 6 символів';
                    }
                    return null;
                  },
                ),
                const SizedBox(height: 16),

                // 5. Поле: підтвердження пароля (п. 3, 4, 5 ТЗ)
                TextFormField(
                  controller: _confirmPasswordController,
                  obscureText: _obscureConfirmPassword,
                  style: const TextStyle(color: Colors.white),
                  decoration: _inputDecoration(
                    labelText: 'Підтвердження пароля',
                    hintText: 'Повторіть введений пароль',
                    prefixIcon: Icons.lock_reset_outlined,
                    suffixIcon: IconButton(
                      icon: Icon(
                        _obscureConfirmPassword
                            ? Icons.visibility_outlined
```

```
                          : Icons.visibility_off_outlined,
                      color: const Color(0xFF8B949E),
                    ),
                    onPressed: () {
                      setState(() {
                        _obscureConfirmPassword = !_obscureConfirmPassword;
                      });
                    },
                  ),
                ),
                validator: (value) {
                  if (value == null || value.isEmpty) {
                    return 'Підтвердіть пароль';
                  }
                  if (value != _passwordController.text) {
                    return 'Поле підтвердження пароля повинно збігатися з паролем';
                  }
                  return null;
                },
              ),
              const SizedBox(height: 28),

              // 6. Кнопка «Зареєструватися» (п. 7 ТЗ)
              ElevatedButton(
                style: ElevatedButton.styleFrom(
                  backgroundColor: const Color(0xFFD4FF00),
                  foregroundColor: const Color(0xFF0D0F13),
                  padding: const EdgeInsets.symmetric(vertical: 16),
                  shape: RoundedRectangleBorder(
                    borderRadius: BorderRadius.circular(12),
                  ),
                  elevation: 3,
                ),
                onPressed: _submitForm,
                child: const Row(
                  mainAxisAlignment: MainAxisAlignment.center,
                  children: [
                    Icon(Icons.how_to_reg, size: 20),
                    SizedBox(width: 10),
                    Text(
                      'Зареєструватися',
                      style: TextStyle(
                        fontSize: 16,
                        fontWeight: FontWeight.bold,
                        letterSpacing: 0.4,
                      ),
                    ),
                  ],
                ),
              ),
              const SizedBox(height: 16),

              // Пояснювальний статус
              Container(
                padding: const EdgeInsets.all(12),
                decoration: BoxDecoration(
                  color: const Color(0xFF15181F),
                  borderRadius: BorderRadius.circular(10),
                  border: Border.all(color: const Color(0xFF2A3142)),
                ),
                child: const Row(
                  children: [
                    Icon(Icons.info_outline, color: Color(0xFF4FC3F7), size: 18),
                    SizedBox(width: 8),
                    Expanded(
                      child: Text(
                        'Усі поля перевіряються автоматично. Перехід на головний екран можливий
лише за відсутності помилок.',
```

```
                        style: TextStyle(fontSize: 11, color: Color(0xFF8B949E)),
                      ),
                    ),
                  ],
                ),
              ),
            ],
          ),
        ),
      ),
    ),
  );
  }

  // Загальний стиль оформлення текстових полів
  InputDecoration _inputDecoration({
    required String labelText,
    required String hintText,
    required IconData prefixIcon,
    Widget? suffixIcon,
  }) {
    return InputDecoration(
      labelText: labelText,
      labelStyle: const TextStyle(color: Color(0xFF8B949E), fontSize: 14),
      hintText: hintText,
      hintStyle: const TextStyle(color: Color(0xFF555F70), fontSize: 13),
      prefixIcon: Icon(prefixIcon, color: const Color(0xFF8B949E), size: 20),
      suffixIcon: suffixIcon,
      filled: true,
      fillColor: const Color(0xFF15181F),
      contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
      border: OutlineInputBorder(
        borderRadius: BorderRadius.circular(12),
        borderSide: const BorderSide(color: Color(0xFF2A3142)),
      ),
      enabledBorder: OutlineInputBorder(
        borderRadius: BorderRadius.circular(12),
        borderSide: const BorderSide(color: Color(0xFF2A3142)),
      ),
      focusedBorder: OutlineInputBorder(
        borderRadius: BorderRadius.circular(12),
        borderSide: const BorderSide(color: Color(0xFFD4FF00), width: 1.5),
      ),
      errorBorder: OutlineInputBorder(
        borderRadius: BorderRadius.circular(12),
        borderSide: const BorderSide(color: Color(0xFFFF5252), width: 1.2),
      ),
      focusedErrorBorder: OutlineInputBorder(
        borderRadius: BorderRadius.circular(12),
        borderSide: const BorderSide(color: Color(0xFFFF5252), width: 1.5),
      ),
      errorStyle: const TextStyle(color: Color(0xFFFF5252), fontSize: 12),
    );
  }
}
```

Лістинг файлу test/widget_test.dart (Комплексні віджет-тести валідації та навігації):

```
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:registration_app/main.dart';
import 'package:registration_app/screens/home_screen.dart';
import 'package:registration_app/screens/register_screen.dart';

void main() {
```

```
setUp(() {
  TestWidgetsFlutterBinding.ensureInitialized();
});

void setPhoneSize(WidgetTester tester) {
  tester.view.physicalSize = const Size(800, 1400);
  tester.view.devicePixelRatio = 1.0;
}

testWidgets('1. Відображення екрана реєстрації з усіма 5 полями та кнопкою',
    (WidgetTester tester) async {
  setPhoneSize(tester);
  addTearDown(tester.view.resetPhysicalSize);

  await tester.pumpWidget(const RegistrationApp());
  await tester.pumpAndSettle();

  expect(find.text('Реєстрація'), findsOneWidget);
  expect(find.byType(Form), findsOneWidget);
  expect(find.byType(TextFormField), findsNWidgets(5));
  expect(find.text("Ім'я користувача"), findsOneWidget);
  expect(find.text('Дата народження'), findsOneWidget);
  expect(find.text('Електронна пошта (Email)'), findsOneWidget);
  expect(find.text('Пароль'), findsOneWidget);
  expect(find.text('Підтвердження пароля'), findsOneWidget);
  expect(find.widgetWithText(ElevatedButton, 'Зареєструватися'), findsOneWidget);
});

testWidgets('2. Валідація: спроба відправки порожньої форми блокує перехід і виводить помилки',
    (WidgetTester tester) async {
  setPhoneSize(tester);
  addTearDown(tester.view.resetPhysicalSize);

  await tester.pumpWidget(const RegistrationApp());
  await tester.pumpAndSettle();

  final submitBtn = find.widgetWithText(ElevatedButton, 'Зареєструватися');
  await tester.ensureVisible(submitBtn);
  await tester.tap(submitBtn);
  await tester.pumpAndSettle();

  // Перевіряємо помилки для всіх полів
  expect(find.text("Поле імені користувача не може бути порожнім"), findsOneWidget);
  expect(find.text('Введіть дату народження'), findsOneWidget);
  expect(find.text('Введіть адресу електронної пошти'), findsOneWidget);
  expect(find.text('Введіть пароль'), findsOneWidget);
  expect(find.text('Підтвердіть пароль'), findsOneWidget);

  // HomeScreen НЕ повинен з'явитися (п. 13 ТЗ)
  expect(find.byType(HomeScreen), findsNothing);
});

testWidgets('3. Валідація: некоректні формати даних (довжина, @, паролі)',
    (WidgetTester tester) async {
  setPhoneSize(tester);
  addTearDown(tester.view.resetPhysicalSize);

  await tester.pumpWidget(const RegistrationApp());
  await tester.pumpAndSettle();

  final textFields = find.byType(TextFormField);

  // 1. Коротке ім'я (< 3 символів)
  await tester.enterText(textFields.at(0), 'An');

  // 2. Некоректна дата
  await tester.enterText(textFields.at(1), 'не_дата');
```

```
    // 3. Email без @
    await tester.enterText(textFields.at(2), 'invalid-email');

    // 4. Пароль менше 6 символів
    await tester.enterText(textFields.at(3), '123');

    // 5. Підтвердження пароля не збігається
    await tester.enterText(textFields.at(4), '999999');

    final submitBtn = find.widgetWithText(ElevatedButton, 'Зареєструватися');
    await tester.ensureVisible(submitBtn);
    await tester.tap(submitBtn);
    await tester.pumpAndSettle();

    expect(find.text("Ім'я користувача повинно містити щонайменше 3 символи"), findsOneWidget);
    expect(find.text('Некоректний формат дати (приклад: 2004-05-15 або 15.05.2004)'),
findsOneWidget);
    expect(find.text('Email повинен містити символ @'), findsOneWidget);
    expect(find.text('Пароль повинен містити не менше 6 символів'), findsOneWidget);
    expect(find.text('Поле підтвердження пароля повинно збігатися з паролем'), findsOneWidget);

    expect(find.byType(HomeScreen), findsNothing);
  });

  testWidgets('4. Успішна валідація та перехід на HomeScreen із передачею імені',
      (WidgetTester tester) async {
    setPhoneSize(tester);
    addTearDown(tester.view.resetPhysicalSize);

    await tester.pumpWidget(const RegistrationApp());
    await tester.pumpAndSettle();

    final textFields = find.byType(TextFormField);

    await tester.enterText(textFields.at(0), 'Андрій');
    await tester.enterText(textFields.at(1), '2004-05-15');
    await tester.enterText(textFields.at(2), 'andrii@example.com');
    await tester.enterText(textFields.at(3), 'secret123');
    await tester.enterText(textFields.at(4), 'secret123');

    final submitBtn = find.widgetWithText(ElevatedButton, 'Зареєструватися');
    await tester.ensureVisible(submitBtn);
    await tester.tap(submitBtn);
    await tester.pumpAndSettle();

    // Перевіряємо перехід на HomeScreen (п. 8 ТЗ)
    expect(find.byType(HomeScreen), findsOneWidget);

    // Перевіряємо текст привітання з переданим ім'ям (п. 9, 10 ТЗ)
    expect(find.text('Вітаємо, Андрій! 🎉'), findsOneWidget);
    expect(find.text('andrii@example.com'), findsOneWidget);
    expect(find.text('2004-05-15'), findsOneWidget);

    // Перевіряємо кнопку повернення
    final backBtn = find.widgetWithText(OutlinedButton, 'Повернутися до реєстрації');
    await tester.ensureVisible(backBtn);
    await tester.tap(backBtn);
    await tester.pumpAndSettle();

    expect(find.byType(RegisterScreen), findsOneWidget);
  });

  testWidgets('5. Кнопка автозаповнення демо-даних заповнює валідну форму',
      (WidgetTester tester) async {
    setPhoneSize(tester);
    addTearDown(tester.view.resetPhysicalSize);

    await tester.pumpWidget(const RegistrationApp());
```

```
    await tester.pumpAndSettle();

    // Тиснемо кнопку автозаповнення в AppBar
    await tester.tap(find.byIcon(Icons.auto_fix_high));
    await tester.pumpAndSettle();

    expect(find.text('Андрій Багрій-Белз'), findsOneWidget);
    expect(find.text('2004-05-15'), findsOneWidget);
    expect(find.text('andrii.bagriy@nulp.ua'), findsOneWidget);

    // Натискаємо «Зареєструватися»
    final submitBtn = find.widgetWithText(ElevatedButton, 'Зареєструватися');
    await tester.ensureVisible(submitBtn);
    await tester.tap(submitBtn);
    await tester.pumpAndSettle();

    expect(find.byType(HomeScreen), findsOneWidget);
    expect(find.text('Вітаємо, Андрій Багрій-Белз! 🎉'), findsOneWidget);
  });
}
```

2 Результат виконання коду

Роботу застосунку було протестовано як за допомогою автоматизованого тестового набору Flutter (flutter test, успішно виконано 5 тестів), так і безпосередньо на віртуальному мобільному пристрої Android (Google Pixel, API 36). Було перевірено поведінку інтерфейсу при спробі відправки порожніх або некоректно заповнених полів, відображення індивідуальних червоних повідомлень валідатора, а також успішну реєстрацію з переходом на HomeScreen та коректне виведення тексту привітання: «Вітаємо, Андрій Багрій-Белз! 🎉».

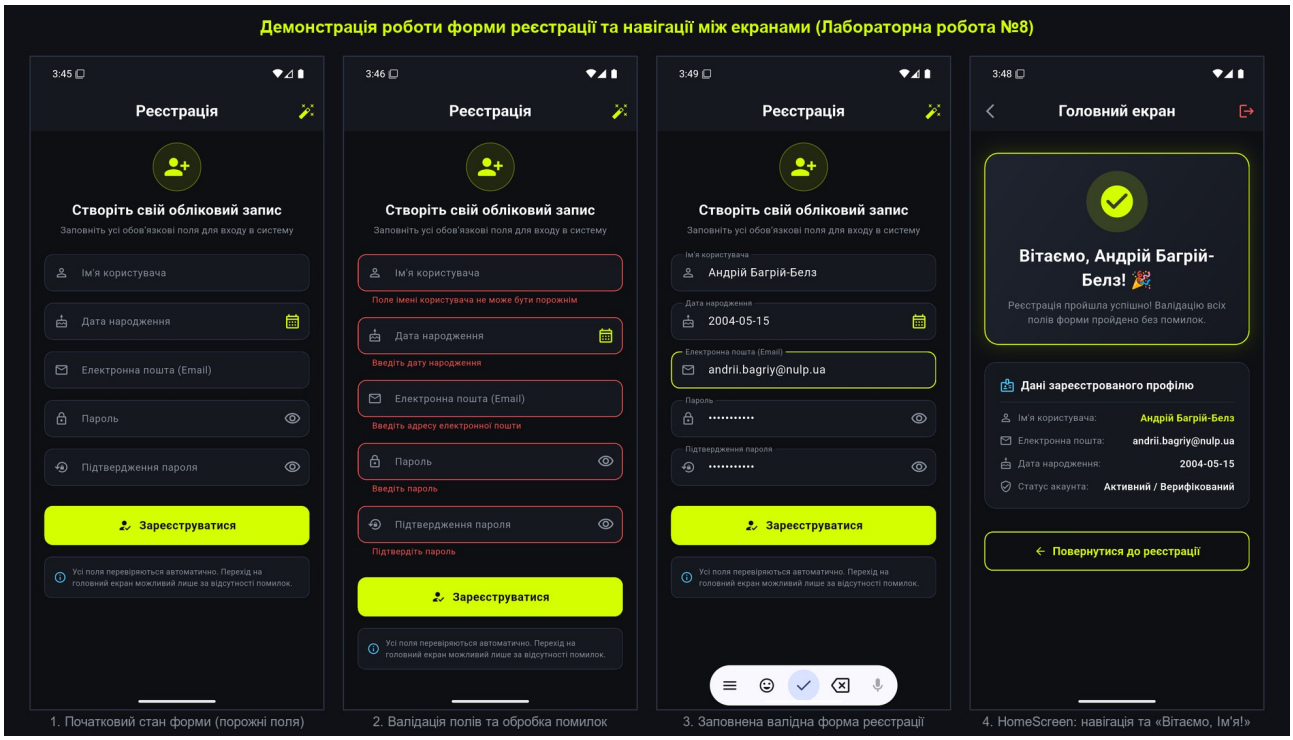


Рисунок 1 — Демонстрація роботи форми реєстрації: початковий стан, обробка помилок валідації, заповнення та перехід на HomeScreen

Контрольні запитання

1. Для чого у Flutter використовується віджет Form?

Віджет Form є контейнером для групування та централізованого керування станом множини пов'язаних полів введення (таких як TextFormField). Він забезпечує єдиний життєвий цикл для всієї форми, дозволяючи одночасно запускати валідацію всіх дочірніх полів за допомогою методу validate(), зберігати значення через save(), скидати форму до початкового стану за допомогою reset(), а також відстежувати зміни та взаємодію користувача через колбек onChanged. Завдяки Form логіка перевірки не розпорошується по окремих обробниках, а виконується декларативно та координовано через пов'язаний ключ GlobalKey<FormState>.

2. У чому різниця між TextField і TextFormField?

TextField — це низькорівневий базовий віджет для текстового введення користувача, який не має вбудованої інтеграції з віджетом Form і не підтримує стандартний механізм валідації через функцію-валідатор. Натомість TextFormField — це спеціалізована обгортка над TextField, яка розширює клас FormField<String> і повністю інтегрована в екосистему Form. TextFormField містить властивість validator: (value) { ... }, яка повертає рядок помилки або null, властивості onSaved та initialValue, а також автоматично реагує на виклики FormState.validate(), відображаючи повідомлення про помилку червоним кольором та змінюючи стан рамки без необхідності ручного оновлення UI.

3. Для чого використовується TextEditingController?

TextEditingController — це клас контролера з Flutter SDK, який слугує програмним мостом між бізнес-логікою застосунку та внутрішнім станом текстового поля введення (TextField або TextFormField). Він надає такі ключові можливості: 1) Читання поточного значення тексту (controller.text); 2) Програмна зміна або очищення тексту (controller.text = '...', controller.clear()); 3) Керування позицією курсора та виділенням тексту (controller.selection); 4) Прослуховування змін тексту в режимі реального часу за допомогою патерну слухача (controller.addListener(() { ... })). Оскільки контролер реєструє нативні слухачі та підтримує внутрішній стан, він вимагає обов'язкового виклику dispose() у методі життєвого циклу dispose() віджета.

4. Яке призначення методу validate()?

Метод validate() належить класу FormState (доступ до якого отримують через _formKey.currentState!.validate()). Його призначення полягає в послідовному рекурсивному обході всіх дочірніх віджетів типу FormField (зокрема TextFormField), зареєстрованих у даній формі, та виклику їхніх функцій validator(value). Якщо кожен валідатор повертає null (помилок немає), метод validate() повертає true. Якщо хоча б один валідатор повертає текстовий рядок із повідомленням про помилку, метод повертає false, а відповідні поля автоматично перемальовуються з червоним текстом помилки під полем та зміненим стилем межі (errorBorder). Це дозволяє заблокувати перехід на наступний екран, якщо форма заповнена некоректно.

5. Для чого потрібен GlobalKey<FormState>?

GlobalKey<FormState> — це унікальний глобальний ключ у дереві віджетів Flutter, який забезпечує доступ до екземпляра стану FormState віджета Form з будь-якого місця класу (наприклад, з обробника натискання кнопки onPressed або зовнішніх методів). Оскільки

стан Form зберігається у внутрішньому StatefulElement, передача ключа key: _formKey у конструктор Form дає розробнику можливість викликати методи цього стану: _formKey.currentState!.validate(), _formKey.currentState!.save() або _formKey.currentState!.reset(). Крім того, GlobalKey гарантує збереження стану форми під час перебудови та переміщення віджетів у дереві.

6. Як реалізувати перевірку порожнього поля у формі?

Перевірка порожнього поля реалізується у властивості validator віджета TextFormField. Функція валідатора отримує поточне значення value типу String? та перевіряє його на null та порожнечу з урахуванням пробільних символів: validator: (value) { if (value == null || value.trim().isEmpty) { return 'Поле не може бути порожнім'; } return null; }. Використання методу trim() є критично важливим, оскільки введення лише пробілів або символів табуляції не вважається валідним заповненням текстового поля. Повернення null сигналізує системі валідації про успішне проходження перевірки.

7. Для чого використовується властивість obscureText?

Властивість obscureText (булеве значення bool, за замовчуванням false) у віджетах TextField та TextFormField використовується для безпечного приховування символів, що вводяться користувачем. Найчастіше вона застосовується для полів введення паролів, PIN-кодів, CVV-кодів банківських карток або секретних ключів. Коли obscureText: true, введені літери замінюються на маскуючі маркери (крапки або зірочки), запобігаючи підгляданню сторонніми особами (shoulder surfing). У сучасних UI зазвичай додають перемикач видимості пароля (IconButton із зображенням ока в suffixIcon), який інвертує стан змінної _obscureText через setState().

8. Який віджет найчастіше використовується для кнопки входу?

Для кнопки відправки форми, реєстрації чи входу найчастіше використовується ElevatedButton (або FilledButton у специфікації Material 3). Цей віджет є акцентною тривимірною кнопкою з помітним кольоровим фоном і тінню (elevation), яка візуально привертає увагу користувача як основний заклик до дії (Call to Action — CTA). Він містить властивість onPressed, яка прив'язується до функції валідації форми та відправки даних, а також властивість child (зазвичай Text або Row з іконкою та текстом).

9. Яке призначення Navigator.push()?

Метод Navigator.push(BuildContext context, Route route) використовується для переходу на новий екран шляхом розміщення нового маршруту (найчастіше MaterialPageRoute) на вершину стека навігації застосунку. При цьому попередній екран залишається у пам'яті під новим екраном у стеку. На новому екрані в AppBar автоматично додається кнопка повернення назад (<-), натискання на яку або виклик Navigator.pop(context) знімає поточний екран зі стека та повертає користувача до попереднього стану форми без втрати введених даних.

10. У чому різниця між Navigator.push() і Navigator.pushReplacement()?

Різниця полягає у поведінці стека маршрутів навігації: 1) Navigator.push() додає новий маршрут на вершину поточного стека, зберігаючи всі попередні екрани під ним. Користувач може легко повернутися назад кнопкою «Back» або викликом pop(). 2) Navigator.pushReplacement() видаляє (замінює) поточний екран на вершині стека новим маршрутом. Попередній екран вивантажується, і користувач не може повернутися на

нього натисканням системної кнопки «Назад». Це стандартний патерн для авторизації: після успішного входу або реєстрації екран логіну замінюється головним екраном застосунку (HomeScreen), щоб запобігти випадковому поверненню користувача на форму введення облікових даних.

11. Як передати дані з одного екрана на інший?

Найпоширенішим і найчистішим способом передачі даних між екранами у Flutter є використання параметрів конструктора цільового класу-віджета. Цільовий екран (наприклад, HomeScreen) оголошує необхідні поля у своєму класі (наприклад, final String username;) та вимагає їх у конструкторі (required this.username). Під час навігації екран-відправник створює маршрут та інстанціює цільовий віджет, передаючи значення безпосередньо в конструктор: Navigator.push(context, MaterialPageRoute(builder: (context) => HomeScreen(username: _usernameController.text.trim()))). Крім цього способу, дані можна передавати через RouteSettings(arguments: ...) для іменованих маршрутів (Navigator.pushNamed), або використовуючи інструменти керування станом (Provider, Riverpod, Bloc) та InheritedWidget для глобального контексту.

12. Що станеться, якщо не викликати dispose() для контролерів?

Якщо не викликати dispose() для об'єктів TextEditingController (або FocusNode, AnimationController, StreamSubscription), це призведе до витоку пам'яті (memory leak). Контролер реєструє слухачів та нативні ресурси у внутрішній системі Flutter і платформі ОС. Коли стан віджета видаляється з дерева (State.dispose()), відсутність виклику controller.dispose() залишає контролер і його слухачів у пам'яті як активні об'єкти. Слухачі продовжуватимуть тримати посилання на екземпляр State і його ресурси, перешкоджаючи збирачу сміття (Garbage Collector) звільнити пам'ять. При повторному переході на екран створюватимуться нові контролери, що поступово збільшуватиме споживання оперативної пам'яті, призводитиме до падіння продуктивності, пропущених кадрів (jank) і потенційного аварійного завершення роботи застосунку (crash через Out of Memory).

Висновок

На цій лабораторній роботі, я сформував ґрунтовні практичні навички роботи з формами введення даних у фреймворку Flutter, опанував взаємодію між віджетами Form, TextFormField та класом TextEditingController, реалізував централізовану перевірку правильності введених даних (валідацію за допомогою GlobalKey<FormState> та методу validate()), організував вибір дати за допомогою нативного діалогу showDatePicker, а також побудував навігаційний потік між екранами RegisterScreen та HomeScreen із гарантованою передачею імені користувача та забороною переходу у разі виявлення помилок заповнення форми. Коректність роботи логіки валідації та міжвіконних переходів підтверджено повним набором автоматизованих віджет-тестів та верифікацією на емуляторі Android.